Building a Simple Snake Game

V. Markos

Abstract

This is a simple homework half-baked project where you are goind to build a fully functional terminal-based snake game using Python.

Contents

Introduction **1**

Project Structure **2**

Snakes, Food et al **2**

Exercise 1 2
Exercise 2 3
Exercise 3 3
Exercise 4 3

Testing and Extending **3**

Introduction

In this homework exercises, you are going to build a simple snake game using Python and simple data structures explored today in class. In particular, you are going to model a snake by a Python list of points on a 2D game grid and also use some code (provided with this document) to listen to user keyboard input.

> **Important note for Windows users.** While `curses` is a built-in Python library, it is not normally distributed with Windows Python installation, thus yo have to install a similar dependeincy on your own. To do so:
>
> - Open a cmd / PowerShell window.
> - Run `pip install windows-curses` or, if that fails, `pip3 install windows-curses`.

- Wait for the installation process to complete and you should be ready!

Project Structure

Within this project you can find the following files:

```
main.py
snake.py
utils.py
```

Each of those has a particular purpose:

- `main.py` will serve as the main entry point of your project. i.e., this is the file you will run your project from. **You you should not modify this file!**
- `snake.py` will contain some snake-related utilities. You have to **fill code** in there, too.
- `keyboard.py` contains keyboard related utilities, mostly for listening to key presses. **You should not modify this file!**

While you do not have to fill code in all files, you are suggested to read through all files in case you need to seek some further information about the underlying implementation.

Snakes, Food et al

Before you start implementing anything, you should consider what data structures you are going to use. The two key objects in this game are:

- the snake itself, and;
- the food collection, corresponding to all foods.

Another important entity are grid points, of which everything else comprises. Within the source files shared, it is suggested that you use 2-tuples of floats to represent those, but, of course, you can choose any other way you see fit. After you have concluded to the data structures you are going to usem start considering the following function implementations.

Exercise 1

Implement the function `cuts_self()` which determines whether a sanke cuts itself. This might be easy enough; it can even be a one-liner.

Exercise 2

Implement the function `move_snake()`, which updates the snake data structure and the corresponding food collection by appropriately updating both. Make sure to figure out what happens in the case when the snake consumes a food point on the grid.

In order to implement the above function you might find it useful to have a separate function like `next_point()` which, given a certain point and a move, computes and returns the corresponding point when applying the corresponding move on that point. So, also implement this function, if needed.

Exercise 3

How it all ends? Well, in this particular case, this is quite easy, so, imlpementing `is_game_over()` should also be a simple task.

Exercise 4

A bit counter-intuitive, but since we have implemented all meaningful aspects of our game logic, we now need to wrap things up in a single `init_game()` function, which will compute and return some initial values for the snake, food and first move / moving direction data structures. This might be just a series of sequential decisions, so no deep logic, just a few decisions to be make.

Testing and Extending

Having implemented your game, it is now high time to test it by playing it! Do all moving commands (`wasd`) work as expected? Are there any bugs? How to address them?

Also, once you have completed and tested your game, you can also think of ways to extend it. Maybe show score on screen or at least keep track of it and print it once the game is over? Or, maybe add different food particles of different value to the player, e.g., some food particles increase snake length by 1 block, while “super-food” increases it length, e.g., by 2 or 3. Let your creativity surprise you!

Have fun!